B.Sc.(CS) I Year

Web Technology

Compiled By Mr. Dahale S.V

B.Sc.(CS) I Year

NOTES of WEB TECHNOLOGY

FY B.Sc.
(Computer Science)

Syllabus of Web Technology

UNIT – I

1. Introduction of HTML Documents

- Historical Roots of HTML,
- Web page, Website,
- Structure of HTML documents and Basic Tags: HTML, HEAD, TITLE, ODY
- Formatting Tags: Paragraph Tags, List tags, HR Tag.
- Headings Tags, PRE tag, DIV tag, SPAN tag.
- FONT Tag, ADDRESS tag, MARQUEE tag.
- Text-Level Elements & other different formatting tags.

UNIT – II

2. Technologies for Web Application

- WWW, Web browser.
- U.R.L. concept.
- Web server, Web protocols: HTTP, FTP, Telnet.
- Hyperlink (Anchor) Tag & it's all attributes,
- Creating Email Hyperlinks (using mail to anchor)
- The Role of Images on the Web, tag & it's all attributes, Using Images as links.
- Tables in HTML:- TABLE, TR, TH, TD tag with example, table with all Attributes

UNIT – III

3. Basic Interactivity and DHTML

- Frames in HTML: FRAMESET & FRAME tags & its attributes
- Simple Frame Example. Forms in HTML: Introduction to forms.
- FORM element & it's attributes (Action, Method (GET, POST), Name)
- Form controls: Text Controls, Password Field, Multiline Text Input,
- Pull-Down Menus, Check Box, Radio Buttons, Scrolled List,
- Reset Button and Submit button.

- Introduction of DHTML, Ramifications of DHTML
- Rollover Buttons.

UNIT – IV

4. CSS and Java Script

- Introduction to Cascading Style Sheets
- Embedded Styles, Inline Styles, Imported/External Styles.
- Introduction of JAVA Script
- Adding script to documents with example. Variables.
- Input and Output statements of JAVA Script

Unit-I Introduction to HTML Documents

Historical Roots of HTML

When you consider why HTML is the way that it is, ask a simple question: do you know for sure what kind of computers, screens, or browser types people viewing your Web page have? The answer is no. So many different screen sizes, operating systems, color palettes, and other factors exist that creating software on all systems would be a nightmare. Imagine, then, the problem faced by Tim Berners-Lee, a researcher at the Conseil Europeen pour Recherche Nucleaire (CERN) laboratory in Geneva, Switzerland. In 1989, Berners-Lee had the taskof creating a hypertext delivery environment that could be used as an interface to scientific information, and that could render this information equally well on Macintosh systems with small screens, NeXT Workstations, IBM PCs, and a variety of other platforms.

Rather than give up because of the variations in screen support, Berners-Lee developed the first versions of HTML, opting to concentrate on providing the content and structure first and worry about the presentation later. This made sense, because the group for whom he was designing the environment consisted of scientists looking at technical information—hardly a group looking for the latest in fonts and graphic design techniques. The presentation would be left up to the browser.

The HTML language eventually was defined as an application of **Standard Generalized Markup Language (SGML), which serves as a base for defining markup languages.** Much of the flavor of HTML as a structured language (instead of a presentation language) comes from this relationship with SGML. Nothing indicates that the original designers of the Web didn't care about presentation—plenty of evidence suggests that they did.

But the project was intended to evolve over time and fundamental issues, such as linking, structure, and network delivery, had to be resolved first. **Deployed by late 1991**, the Web grew slowly at first. In its infancy, it was characterized by a textual interface that was unattractive and somewhat difficult to use. However, much of the infrastructure necessary to

make the Web work— including basic HTML, HTTP, and MIME—was in place long before the Web took off.

WEBSITE

A **website**, also written as **web site**, is a collection of related web pages, including multimedia content, typically identified with a common domain name, and published on at least one web server. A web site may be accessible via a public Internet Protocol (IP) network, such as the Internet, or a private local area network (LAN), by referencing a uniform resource locator (URL) that identifies the site.

Websites have many functions and can be used in various fashions; a website can be a personal website, a commercial website for a company, a government website or a non-profit organization website. Websites can be the work of an individual, a business or other organization, and are typically dedicated to a particular topic or purpose, ranging from entertainment and social networking to providing news and education. All publicly accessible websites collectively constitute the World Wide Web, while private websites, such as a company's website for its employees, are typically a part of an intranet.

Web pages, which are the building blocks of websites, are documents, typically composed in plain text interspersed with formatting instructions of Hypertext Markup Language (HTML, XHTML). They may incorporate elements from other websites with suitable markup anchors. Web pages are accessed and transported with the Hypertext Transfer Protocol(HTTP), which may optionally employ encryption (HTTP Secure, HTTPS) to provide security and privacy for the user. The user's application, often a web browser, renders the page content according to its HTML markup instructions onto a display terminal.

Hyper linking between web pages conveys to the reader the site structure and guides the navigation of the site, which often starts with a home page containing a directory of the site web content. Some websites require user registration or subscription to access content. Examples of subscription websites include many business sites, parts of news websites, academic journal websites, gaming websites, file-sharing websites, message boards, web-based email, social networking websites, websites providing real-time stock market data, as well as sites providing various other services. As of 2016, end users can access websites on a range of

devices, including desktop and laptop computers, tablet computers, smart phones and smart TVs.

A **web page** (US spelling **webpage** or **Web page**) is a document that is suitable for the World Wide Web and web browsers. A web browser displays a web page on amonitor or mobile device. The web page is what displays, but the term also refers to a computer file, usually written in HTML or comparable markup language. Web browsers coordinate the various web resource elements for the written web page, such as style sheets, scripts, and images, to present the web page.

Typical web pages provide hypertext that includes a navigation bar or a sidebarmenu to *other* web pages via hyperlinks, often referred to as *links*.

On a network, a web browser can retrieve a web page from a remote web server. On a higher level, the web server may restrict access to only a private network such as a corporate intranet or it provides access to the World Wide Web. On a lower level, the web browser uses the Hypertext Transfer Protocol (HTTP) to make such requests.

A *static web page* is delivered exactly as stored, as web content in the web server's file system, while a *dynamic web page*is generated by a web application that is driven by server-side software or client-side scripting. Dynamic website pages help the browser (the client) to enhance the web page through user input to the server.

The Phases of Web Site Development

The following are the Six basic phases of Web site development:

- Information Gathering/Analysis: Collect all the necessary information from user regarding website.
- Planning:

Setting goals, specifying content, organizing content, and setting the user interface to navigate content.

Planning is an objective of each and every activity, where we want to discover things that belong to the project. An important task in creating a software program is extracting the requirements or requirements analysis from Customers. Typically customer have an abstract idea of what they want as an end result but do not know what *software* should do.

Once the general requirements are gathered from the client, an analysis of the scope of the development should be determined and clearly stated. This is often called a scope document.

- Design:

Once the requirements are established, the design of the software can be established in a software design document. This involves a preliminary, or high-level design of the main modules with an overall picture (such as a block diagram) of how the parts fit together. The language, operating system, and hardware components should all be known at this time. Then a detailed or low-level design is created, perhaps with prototyping as proof-of-concept or to firm up requirements.

In this phase Creating content, implementing navigation and the user interface, and coding the site, which may include HTML, programming, and database development is decided.

- Coding:

After the design is get finalized coding is done. Implementation is the part of the process where software engineers actually program the code for the project.

- Testing:

Software testing is an integral and important phase of the software development process. This part of the process ensures that defects are recognized as soon as possible. In some processes, generally known as test-driven development, tests may be developed just before implementation and serve as a guide for the implementation's correctness.Also in this phase browser, and system testing is also chacked for correctness.

- Maintenance :

Maintaining and enhancing software to cope with newly discovered faults or requirements can take substantial time and effort, as missed requirements may force redesign of the software.

Maintaining and updating the site, questioning old goals, and returning to the planning stage Each phase listed has distinct subphases that can be expanded to provide more detail.

The Structure of HTML

The Structure of HTML Documents Regardless of document content, all well-written HTML documents share a common structure. Figure 3-2 provides a template for this. An HTML document begins with a declaration, indicating the version of HTML used by the document. Following this, the element encloses the actual document.

It contains two primary sections, the head and the body, enclosed by the and elements, respectively. The head can contain identifying and other supplementary information about the document, or meta-information. The head always contains the document's title, enclosed by the ...Other supplementary information goes here....

HTML stands for **Hypertext Markup Language**, and it is the most widely used language to write Web Pages.

- **Hypertext** refers to the way in which Web pages (HTML documents) are linked together. Thus the link available on a webpage are called Hypertext.
- As its name suggests, HTML is a **Markup Language**which means you use HTML to simply "mark up" a text document with tags that tell a Web browser how to structure it to display.

Originally, HTML was developed with the intent of defining the structure of documents like headings, paragraphs, lists, and so forth to facilitate the sharing of scientific information between researchers.

Now, HTML is being widely used to format web pages with the help of different tags available in HTML language.

Basic HTML Document

In its simplest form, following is an example of an HTML document:

```
<!DOCTYPE html>
<html>
<head>
<title>This is document title</title>
</head>
<body>
<h1>This is a heading</h1>
<p>Document content goes here.....</p>
</body>
</html>
```

The HTML <title> Tag

The HTML <title> tag is used for specifying the title of the HTML document. Following is an example to give a title to an HTML document:

```
<!DOCTYPE html>
<html>
<head>
<title>HTML Title Tag Example</title>
</head>
<body>
<p>Hello, World!</p>
</body>
</html>
```

Output:

This will produce following result:

Hello, World!

The HTML <head> tag is used for indicating the head section of the HTML document. Tags included inside head tags are not displayed on browser window.

Example

```
<!DOCTYPE html>
<html>
<head>
<title>HTML head Tag</title>
</head>
<body>
actual content goes here
</body>
</html>
```

This will produce following result:

actual content goes here

The HTML <body> tag is used for indicating the main content section of the HTML document. The body tag is placed between the and the tags.

Example

```
<!DOCTYPE html>
<html>
<head>
<title>HTML body Tag</title>
</head>
<body>
Body of the document...
</body>
```

```
</html>
```

This will produce following result:

Body of the document...

Formatting Tags: Paragraph Tags, List tags, HR Tag.

Paragraph Tag

The ** ** tag offers a way to structure your text into different paragraphs. Each paragraph of text should go in between an opening and a closing tag as shown below in the example:

Example

```
<!DOCTYPE html>
<html>
<head>
<title>Paragraph Example</title>
</head>
<body>
<p>Here is a first paragraph of text.</p>
<p>Here is a second paragraph of text.</p>
<p>Here is a third paragraph of text.</p>
</body>
</html>
```

This will produce following result:

Here is a first paragraph of text.

Here is a second paragraph of text.

Here is a third paragraph of text.

2333333333333333333333

HTML offers web authors three ways for specifying lists of information. All lists must contain one or more list elements. Lists may contain −

- **<ul>** − An unordered list. This will list items using plain bullets.
- **<ol>** − An ordered list. This will use different schemes of numbers to list your items.
- **<dl>** − A definition list. This arranges your items in the same way as they are arranged in a dictionary.

HTML Unordered Lists

An unordered list is a collection of related items that have no special order or sequence. This list is created by using HTML **<ul>** tag. Each item in the list is marked with a bullet.

Example

```
<!DOCTYPE html>
<html>

  <head>
    <title>HTML Unordered List</title>
  </head>

  <body>
    <ul>
      <li>Beetroot</li>
      <li>Ginger</li>
      <li>Potato</li>
      <li>Radish</li>
    </ul>
```

```
  </body>

</html>
```

This will produce the following result –

- Beetroot
- Ginger
- Potato
- Radish

The type Attribute

You can use **type** attribute for <ul> tag to specify the type of bullet you like. By default, it is a disc. Following are the possible options −

<ul type = "square">

<ul type = "disc">

<ul type = "circle">

Example

Following is an example where we used <ul type = "square">

```
<!DOCTYPE html>
<html>

  <head>
    <title>HTML Unordered List</title>
  </head>

  <body>
```

```
    <ul type = "square">
      <li>Beetroot</li>
      <li>Ginger</li>
      <li>Potato</li>
      <li>Radish</li>
    </ul>
  </body>

</html>
```

This will produce the following result –

- Beetroot
- Ginger
- Potato
- Radish

Example

Following is an example where we used <ul type = "disc"> −

```
<!DOCTYPE html>
<html>

  <head>
    <title>HTML Unordered List</title>
  </head>

  <body>
    <ul type = "disc">
      <li>Beetroot</li>
      <li>Ginger</li>
      <li>Potato</li>
```

```
      <li>Radish</li>
    </ul>
  </body>

</html>
```

This will produce the following result –

- Beetroot
- Ginger
- Potato
- Radish

Example

Following is an example where we used <ul type = "circle"> −

Live Demo

```
<!DOCTYPE html>
<html>

  <head>
    <title>HTML Unordered List</title>
  </head>
  <body>
    <ul type = "circle">
      <li>Beetroot</li>
      <li>Ginger</li>
      <li>Potato</li>
      <li>Radish</li>
    </ul>
  </body>

</html>
```

This will produce the following result –

- Beetroot
- Ginger
- Potato
- Radish

HTML Ordered Lists

If you are required to put your items in a numbered list instead of bulleted, then HTML ordered list will be used. This list is created by using **<ol>** tag. The numbering starts at one and is incremented by one for each successive ordered list element tagged with <li>.

Example

```
<!DOCTYPE html>
<html>

  <head>
    <title>HTML Ordered List</title>
  </head>

  <body>
    <ol>
      <li>Beetroot</li>
      <li>Ginger</li>
      <li>Potato</li>
      <li>Radish</li>
    </ol>
  </body>

</html>
```

This will produce the following result –

1. Beetroot
2. Ginger
3. Potato
4. Radish

The type Attribute

You can use **type** attribute for <ol> tag to specify the type of numbering you like. By default, it is a number. Following are the possible options −

<ol type = "1"> - Default-Case Numerals.

<ol type = "I"> - Upper-Case Numerals.

<ol type = "i"> - Lower-Case Numerals.

<ol type = "A"> - Upper-Case Letters.

<ol type = "a"> - Lower-Case Letters.

Example

Following is an example where we used <ol type = "1">

```
<!DOCTYPE html>
<html>

  <head>
    <title>HTML Ordered List</title>
  </head>

  <body>
    <ol type = "1">
      <li>Beetroot</li>
      <li>Ginger</li>
      <li>Potato</li>
      <li>Radish</li>
```

```
    </ol>
  </body>

</html>
```

This will produce the following result –

1. Beetroot
2. Ginger
3. Potato
4. Radish

Example

Following is an example where we used <ol type = "I">

```
<!DOCTYPE html>
<html>

  <head>
    <title>HTML Ordered List</title>
  </head>

  <body>
    <ol type = "I">
      <li>Beetroot</li>
      <li>Ginger</li>
      <li>Potato</li>
      <li>Radish</li>
    </ol>
  </body>

</html>
```

This will produce the following result –

I. Beetroot
II. Ginger
III. Potato
IV. Radish

Example

Following is an example where we used <ol type = "i">

Live Demo

```
<!DOCTYPE html>
<html>

  <head>
    <title>HTML Ordered List</title>
  </head>
  <body>
    <ol type = "i">
      <li>Beetroot</li>
      <li>Ginger</li>
      <li>Potato</li>
      <li>Radish</li>
    </ol>
  </body>
</html>
```

This will produce the following result –

i. Beetroot
ii. Ginger
iii. Potato
iv. Radish

Example

Following is an example where we used <ol type = "A" >

```
<!DOCTYPE html>
<html>

  <head>
    <title>HTML Ordered List</title>
  </head>

  <body>
    <ol type = "A">
      <li>Beetroot</li>
      <li>Ginger</li>
      <li>Potato</li>
      <li>Radish</li>
    </ol>
  </body>

</html>
```

This will produce the following result –

A. Beetroot
B. Ginger
C. Potato
D. Radish

Example

Following is an example where we used <ol type = "a">

```
<!DOCTYPE html>
<html>

  <head>
    <title>HTML Ordered List</title>
  </head>

  <body>
    <ol type = "a">
      <li>Beetroot</li>
      <li>Ginger</li>
      <li>Potato</li>
      <li>Radish</li>
    </ol>
  </body>

</html>
```

This will produce the following result –

a. Beetroot
b. Ginger
c. Potato
d. Radish

The start Attribute

You can use **start** attribute for <ol> tag to specify the starting point of numbering you need. Following are the possible options −

<ol type = "1" start = "4"> - Numerals starts with 4.

<ol type = "I" start = "4"> - Numerals starts with IV.

<ol type = "i" start = "4"> - Numerals starts with iv.

<ol type = "a" start = "4"> - Letters starts with d.

<ol type = "A" start = "4"> - Letters starts with D.

Example

Following is an example where we used <ol type = "i" start = "4" >

```
<!DOCTYPE html>
<html>

  <head>
    <title>HTML Ordered List</title>
  </head>

  <body>
    <ol type = "i" start = "4">
      <li>Beetroot</li>
      <li>Ginger</li>
      <li>Potato</li>
      <li>Radish</li>
    </ol>
  </body>

</html>
```

This will produce the following result –

iv. Beetroot
v. Ginger
vi. Potato
vii. Radish

HTML Definition Lists

HTML and XHTML supports a list style which is called **definition lists** where entries are listed like in a dictionary or encyclopedia. The definition list is the ideal way to present a glossary, list of terms, or other name/value list.

Definition List makes use of following three tags.

- <dl> − Defines the start of the list
- <dt> − A term
- <dd> − Term definition
- </dl> − Defines the end of the list

Example

```
<!DOCTYPE html>
<html>
  <head>
    <title>HTML Definition List</title>
  </head>
  <body>
    <dl>
      <dt><b>HTML</b></dt>
      <dd>This stands for Hyper Text Markup Language</dd>
      <dt><b>HTTP</b></dt>
      <dd>This stands for Hyper Text Transfer Protocol</dd>
    </dl>
  </body>

</html>
```

This will produce the following result –

HTML

This stands for Hyper Text Markup Language

HTTP

This stands for Hyper Text Transfer Protocol

Horizontal Lines <HR>

Horizontal lines are used to visually break-up sections of a document. The **<hr>** tag creates a line from the current position in the document to the right margin and breaks the line accordingly.

```
<!DOCTYPE html>
<html>

   <head>
      <title>HTML hr Tag</title>
   </head>

   <body>
      <p>This text will be followed by a horizontal line <hr /></p>
   </body>

</html>
```

This will produce the following result –

This text will be followed by a horizontal line

Again **<hr />** tag is an example of the **empty** element, where you do not need opening and closing tags, as there is nothing to go in between them.

The **<hr />** element has a space between the characters **hr** and the forward slash. If you omit this space, older browsers will have trouble rendering the horizontal line, while if you miss the forward slash character and just use **<hr>** it is not valid in XHTML

w

HTML Headings

Headings are defined with the <h1> to <h6> tags.

<h1> defines the most important heading. <h6> defines the least important heading.

Example

```
<h1>This is heading 1</h1>
<h2>This is heading 2</h2>
<h3>This is heading 3</h3>
<h4>This is heading 4</h4>
<h5>This is heading 5</h5>
<h6>This is heading 6</h6>
```

Heading Tags

Any document starts with a heading. You can use different sizes for your headings. HTML also has six levels of headings, which use the elements **<h1>, <h2>, <h3>, <h4>, <h5>, and <h6>**. While displaying any heading, browser adds one line before and one line after that heading.

Example

```
<!DOCTYPE html>
<html>
<head>
<title>Heading Example</title>
</head>
<body>
<h1>This is heading 1</h1>
<h2>This is heading 2</h2>
<h3>This is heading 3</h3>
<h4>This is heading 4</h4>
```

```
<h5>This is heading 5</h5>
<h6>This is heading 6</h6>
</body>
</html>
```

This will produce following result:

This is heading 1

This is heading 2

This is heading 3

This is heading 4

This is heading 5

This is heading 6

tag

The HTML tag is used for defining a section of your document. With the div tag, you can group large sections of HTML elements together and format them with CSS.

The difference between the div tag and the span tag is that the div tag is used with block-level elements whilst the span tag is used with inline elements.

Example

```
<!DOCTYPE html>
<html>
<head>
<title>HTML div Tag</title>
<link rel="stylesheet" href="style2.css">
</head>
<body>
```

```
<div id="contentinfo">
<p>Welcome to our website. We provide tutorials on various subjects.</p>
</div>
</body>
</html>
```

Here is the csss file *style2.css*

```
#contentinfo p {
	line-height: 20px;
		margin: 30px;
		padding-bottom: 20px;
		text-align: justify;
		width: 140px;
		color: red;
}
```

<pre> tag

The HTML <pre> tag is used for indicating preformatted text. The code tag surrounds the code being marked up.

Browsers normally render pre text in a fixed-pitched font, with whitespace in tact, and without word wrap.

Example

```
<!DOCTYPE html>
<html>
<head>
<title>HTML pre Tag</title>
</head>
<body>
<pre>
```

```
    This text is
 in a fixed-pitch
       font, and it preserves
 both    spaces and line breaks
</pre>
</body>
</html>
```

This will produce following result:

```
    This text is
 in a fixed-pitch
       font, and it preserves
 both    spaces and line breaks
```

tag

The HTML tag is used for grouping and applying styles to inline elements.

There is a difference between the span tag and the div tag. The span tag is used with inline elements whilst the div tag is used with block-level content.

Example

```
<!DOCTYPE html>
<html>
<head>
<title>HTML span Tag</title>
</head>
<body>
<p>This is a paragraph <span style="color:#FF0000;">
This is a paragraph</span>
This is a paragraph</p>
```

```
<p><span style="color:#8866ff;">
This is another paragraph</span></p>
</body>
</html>
```

This will produce following result:

This is a paragraph This is a paragraph This is a paragraph

This is another paragraph

<Font>Tag

Fonts play a very important role in making a website more user friendly and increasing content readability. Font face and color depends entirely on the computer and browser that is being used to view your page but you can use HTML tag to add style, size, and color to the text on your website. You can use a tag to set all of your text to the same size, face, and color. The font tag is having three attributes called size, color, and face to customize your fonts. To change any of the font attributes at any time within your webpage, simply use the tag. The text that follows will remain changed until you close with the tag. You can change one or all of the font attributes within one tag.

Set Font Size

You can set content font size using **size** attribute. The range of accepted values is from 1(smallest) to 7(largest). The default size of a font is 3.

Example

```
!DOCTYPE html>
<html>

  <head>
    <title>Setting Font Size</title>
  </head>

  <body>
```

```
    <font size = "1">Font size = "1"</font><br />
    <font size = "2">Font size = "2"</font><br />
    <font size = "3">Font size = "3"</font><br />
    <font size = "4">Font size = "4"</font><br />
    <font size = "5">Font size = "5"</font><br />
    <font size = "6">Font size = "6"</font><br />
    <font size = "7">Font size = "7"</font>
  </body>

</html>
```

This will produce the following result –

Font size = "1"
Font size = "2"
Font size = "3"
Font size = "4"
Font size = "5"
Font size = "6"
Font size = "7"

Relative Font Size

You can specify how many sizes larger or how many sizes smaller than the preset font size should be. You can specify it like **<font size = "+n">** or **<font size = "−n">**

Example

```
<!DOCTYPE html>
<html>

  <head>
    <title>Relative Font Size</title>
  </head>

  <body>
    <font size = "-1">Font size = "-1"</font><br />
    <font size = "+1">Font size = "+1"</font><br />
    <font size = "+2">Font size = "+2"</font><br />
    <font size = "+3">Font size = "+3"</font><br />
```

```
    <font size = "+4">Font size = "+4"</font>
  </body>

</html>
```

This will produce the following result –

Font size = "-1"
Font size = "+1"
Font size = "+2"
Font size = "+3"
Font size = "+4"

Setting Font Face

You can set font face using *face* attribute but be aware that if the user viewing the page doesn't have the font installed, they will not be able to see it. Instead user will see the default font face applicable to the user's computer.

Example

```
<!DOCTYPE html>
<html>

  <head>
    <title>Font Face</title>
  </head>

  <body>
    <font face = "Times New Roman" size = "5">Times New Roman</font><br />
    <font face = "Verdana" size = "5">Verdana</font><br />
    <font face = "Comic sans MS" size =" 5">Comic Sans MS</font><br />
    <font face = "WildWest" size = "5">WildWest</font><br />
    <font face = "Bedrock" size = "5">Bedrock</font><br />
  </body>

</html>
```

This will produce the following result –

Times New Roman
Verdana
Comic Sans MS
WildWest
Bedrock

Specify alternate font faces

A visitor will only be able to see your font if they have that font installed on their computer. So, it is possible to specify two or more font face alternatives by listing the font face names, separated by a comma.

```
<font face = "arial,helvetica">
<font face = "Lucida Calligraphy,Comic Sans MS,Lucida Console">
```

When your page is loaded, their browser will display the first font face available. If none of the given fonts are installed, then it will display the default font face *Times New Roman*.

Setting Font Color

You can set any font color you like using *color* attribute. You can specify the color that you want by either the color name or hexadecimal code for that color.

Note − You can check a complete list of **HTML Color Name with Codes**.

Example

```
<!DOCTYPE html>
<html>

  <head>
    <title>Setting Font Color</title>
  </head>

  <body>
    <font color = "#FF00FF">This text is in pink</font><br />
```

```
    <font color = "red">This text is red</font>
  </body>

</html>
```

This will produce the following result –

This text is in pink
This text is red

The <basefont> Element

The <basefont> element is supposed to set a default font size, color, and typeface for any parts of the document that are not otherwise contained within a <font> tag. You can use the <font> elements to override the <basefont> settings.

The <basefont> tag also takes color, size and face attributes and it will support relative font setting by giving size a value of +1 for a size larger or −2 for two sizes smaller.

```
<!DOCTYPE html>
<html>

  <head>
    <title>Setting Basefont Color</title>
  </head>

  <body>
    <basefont face = "arial, verdana, sans-serif" size = "2" color = "#ff0000">
    <p>This is the page's default font.</p>
    <h2>Example of the &lt;basefont&gt; Element</h2>

    <p><font size = "+2" color = "darkgray">
         This is darkgray text with two sizes larger
       </font>
    </p>

    <p><font face = "courier" size = "-1" color = "#000000">
         It is a courier font, a size smaller and black in color.
       </font>
    </p>
  </body>

</html>
```

This will produce the following result −

This is the page's default font.

Example of the <basefont> Element

This is darkgray text with two sizes larger

It is a courier font, a size smaller and black in color.

Address tag:

The HTML <address> tag is used for indicating an address. The address usually renders in italic.

Example

```
<!DOCTYPE html>
<html>
<head>
<title>HTML address Tag</title>
</head>
<body>
<address>
600 Wisdon Apartments<br />
Filmcity, Kondiura<br />
New Delhi - 50027
</address>
</body>
</html>
```

This will produce following result:

600 Wisdon Apartments

Filmcity, Kondiura

New Delhi -50027

<marquee> tag

The HTML <marquee> tag is used for scrolling piece of text or image displayed either horizontally across or vertically down your web site page depending on the settings.

```
<!DOCTYPE html>
<html>

  <head>
    <title>HTML marquee Tag</title>
  </head>

  <body>
    <marquee>This is basic example of marquee</marquee>
    <marquee direction = "up">The direction of text will be from bottom to top.</marquee>
  </body>

</html>
```

Text level Elements

If you use a word processor, you must be familiar with the ability to make text bold, italicized, or underlined; these are just three of the ten options available to indicate how text can appear in HTML and XHTML.

Bold Text

Anything that appears within **...** element, is displayed in bold as shown below −

Example

```
<!DOCTYPE html>
<html>

  <head>
    <title>Bold Text Example</title>
  </head>

  <body>
    <p>The following word uses a <b>bold</b> typeface.</p>
  </body>

</html>
```

This will produce the following result –

The following word uses a **bold** typeface.

Italic Text

Anything that appears within **...** element is displayed in italicized as shown below −

Example

```
<!DOCTYPE html>
<html>

  <head>
    <title>Italic Text Example</title>
  </head>

  <body>
    <p>The following word uses an <i>italicized</i> typeface.</p>
```

```
  </body>

</html>
```

This will produce the following result –

The following word uses an *italicized* typeface.

Underlined Text

Anything that appears within **<u>...</u>** element, is displayed with underline as shown below −

Example

```
<!DOCTYPE html>
<html>
  <head>
    <title>Underlined Text Example</title>
  </head>
  <body>
    <p>The following word uses an <u>underlined</u> typeface.</p>
  </body>
</html>
```

This will produce the following result –

The following word uses an <u>underlined</u> typeface.

Strike Text

Anything that appears within **<strike>...</strike>** element is displayed with strikethrough, which is a thin line through the text as shown below −

Example

```
<!DOCTYPE html>
<html>

  <head>
    <title>Strike Text Example</title>
  </head>

  <body>
    <p>The following word uses a <strike>strikethrough</strike> typeface.</p>
  </body>

</html>
```

This will produce the following result –

The following word uses a ~~strikethrough~~ typeface.

Monospaced Font

The content of a **<tt>...</tt>** element is written in monospaced font. Most of the fonts are known as variable-width fonts because different letters are of different widths (for example, the letter 'm' is wider than the letter 'i'). In a monospaced font, however, each letter has the same width.

Example

```
<!DOCTYPE html>
<html>

  <head>
    <title>Monospaced Font Example</title>
  </head>
```

```
  <body>
    <p>The following word uses a <tt>monospaced</tt> typeface.</p>
  </body>

</html>
```

This will produce the following result –

The following word uses a `monospaced` typeface.

Superscript Text

The content of a **^{...}** element is written in superscript; the font size used is the same size as the characters surrounding it but is displayed half a character's height above the other characters.

Example

```
<!DOCTYPE html>
<html>

  <head>
    <title>Superscript Text Example</title>
  </head>

  <body>
    <p>The following word uses a <sup>superscript</sup> typeface.</p>
  </body>

</html>
```

This will produce the following result –

The following word uses a ^{superscript} typeface.

Subscript Text

The content of a **_{...}** element is written in subscript; the font size used is the same as the characters surrounding it, but is displayed half a character's height beneath the other characters.

Example

```
<!DOCTYPE html>
<html>

  <head>
    <title>Subscript Text Example</title>
  </head>

  <body>
    <p>The following word uses a <sub>subscript</sub> typeface.</p>
  </body>

</html>
```

This will produce the following result –

The following word uses a $_{\text{subscript}}$ typeface.

Inserted Text

Anything that appears within **<ins>...</ins>** element is displayed as inserted text.

Example

```
<!DOCTYPE html>
<html>
```

```
  <head>
    <title>Inserted Text Example</title>
  </head>

  <body>
    <p>I want to drink <del>cola</del> <ins>wine</ins></p>
  </body>

</html>
```

This will produce the following result –

I want to drink wine

Deleted Text

Anything that appears within **<del>...</del>** element, is displayed as deleted text.

Example

```
<!DOCTYPE html>
<html>

  <head>
    <title>Deleted Text Example</title>
  </head>

  <body>
    <p>I want to drink <del>cola</del> <ins>wine</ins></p>
  </body>

</html>
```

This will produce the following result –

I want to drink wine

Larger Text

The content of the **<big>...</big>** element is displayed one font size larger than the rest of the text surrounding it as shown below −

Example

```
<!DOCTYPE html>
<html>

  <head>
    <title>Larger Text Example</title>
  </head>
  <body>
    <p>The following word uses a <big>big</big> typeface.</p>
  </body>
</html>
```

This will produce the following result –

The following word uses a big typeface.

Smaller Text

The content of the **<small>...</small>** element is displayed one font size smaller than the rest of the text surrounding it as shown below −

Example

```
<!DOCTYPE html>
<html>
```

```
  <head>
    <title>Smaller Text Example</title>
  </head>

  <body>
    <p>The following word uses a <small>small</small> typeface.</p>
  </body>

</html>
```

This will produce the following result –

The following word uses a small typeface.

Grouping Content

The ** ** and **** elements allow you to group together several elements to create sections or subsections of a page.

For example, you might want to put all of the footnotes on a page within a element to indicate that all of the elements within that element relate to the footnotes. You might then attach a style to this element so that they appear using a special set of style rules.

Example

```
<!DOCTYPE html>
<html>

  <head>
    <title>Div Tag Example</title>
  </head>
```

```
  <body>
    <div id = "menu" align = "middle" >
      <a href = "/index.htm">HOME</a> |
      <a href = "/about/contact_us.htm">CONTACT</a> |
      <a href = "/about/index.htm">ABOUT</a>
    </div>

    <div id = "content" align = "left" bgcolor = "white">
      <h5>Content Articles</h5>
      <p>Actual content goes here.....</p>
    </div>
  </body>

</html>
```

This will produce the following result –

HOME | CONTACT | ABOUT

Content Articles

Actual content goes here.....

The element, on the other hand, can be used to group inline elements only. So, if you have a part of a sentence or paragraph which you want to group together, you could use the element as follows.

Example

```
<!DOCTYPE html>
<html>
```

```
  <head>
    <title>Span Tag Example</title>
  </head>

  <body>
    <p>This is the example of <span style = "color:green">span tag</span>
      and the <span style = "color:red">div tag</span> alongwith CSS</p>
  </body>

</html>
```

This will produce the following result −

This is the example of span tag and the div tag alongwith CSS

Unit –II Technologies for Web Applications

What is World Wide Web?

World Wide Web, which is also known as a Web, is a collection of websites or web pages stored in web servers and connected to local computers through the internet. These websites contain text pages, digital images, audios, videos, etc. Users can access the content of these sites from any part of the world over the internet using their devices such as computers, laptops, cell phones, etc. The WWW, along with internet, enables the retrieval and display of text and media to your device.

The building blocks of the Web are web pages which are formatted in HTML and connected by links called "hypertext" or hyperlinks and accessed by HTTP. These links are electronic connections that link related pieces of information so that users can access the desired information quickly. Hypertext offers the advantage to select a word or phrase from text and thus to access other pages that provide additional information related to that word or phrase.

A web page is given an online address called a Uniform Resource Locator (URL). A particular collection of web pages that belong to a specific URL is called a website, e.g., *www.facebook.com*, *www.google.com*, etc. So, the World Wide Web is like a huge electronic book whose pages are stored on multiple servers across the world.

Small websites store all of their WebPages on a single server, but big websites or organizations place their WebPages on different servers in different countries so that when users of a country search their site they could get the information quickly from the nearest server.

So, the web provides a communication platform for users to retrieve and exchange information over the internet. Unlike a book, where we move from one page to another in a sequence, on World Wide Web we follow a web of hypertext links to visit a web page and from that web page to move to other web pages. You need a browser, which is installed on your computer, to access the Web.

web browser:

A **web browser** (commonly referred to as a **browser**) is a software application for accessing information on the World Wide Web. Each individual web page, image, and video is identified by a distinct Uniform Resource Locator (URL), enabling browsers to retrieve these resources from a web server and display them on a user's device.

The capabilities of modern web browsers allow web developers to create highly interactive websites. For example, Ajax enables a browser to dynamically update information on a webpage without the need to reload the page. Advances in CSS allow browsers to display a responsive website layouts and a wide array of visual effects. Cookies allow browsers to remember your settings for specific websites.

A web browser is not the same thing as a search engine, though the two are often confused. For a user, a search engine is just a website, such as google.com, that stores searchable data about other websites. But to connect to a website's server and display its web pages, a user must have a web browser installed on their device.

As of March 2019, more than 4.3 billion people use a browser, which is about 55% of the world's population. The most popular browsers are Chrome, Firefox, Safari, Internet Explorer, and Edge.

While developing a site, we should try to make it compatible to as many browsers as possible. Especially sites should be compatible to major browsers like Explorer, Firefox, Chrome, Netscape, Opera, and Safari.

Functions of web browsers:

- Web Browser is an Application program that displays a World wide web document. It usually uses the internet service to access the document.
- The Web browser requests the server for the web documents and services.
- The web browser acts as an interface between the server and the client and displays a web document to the client.

- The web browser sends an HTTP request and gets an HTTP response.
- Doesn't exist any processing model for the web browser.
- Web browser stores the cookies for different websites.
- The web browser is installed on the client computer.

What Are URLs?

A URL is a uniform way to refer to objects and services on the Internet. Even novice users should be familiar with typing a URL, such as http://www.yahoo.com/, in a browser dialog box to get to a Web site. Internet users use URLs to invoke other Internet services, such as transferring files via FTP or sending e-mail. HTML authors use URLs in their documents to define hyperlinks to other Web documents. Despite its potentially confusing collection of slashes and colons, the URL syntax is designed to provide a clear, simple notation that people can easily understand.

Basic Concepts

To find any arbitrary object on the Internet, you need to find out the following information:

1. First, you need to locate and access the machine on the Internet (or intranet) on which the object resides. Locating the site might be a matter of specifying its domain name or IP address, while accessing the machine might be a matter of providing a username and password.

2. After you access the machine, you need to determine the name of the desired file, where the file is located, and what protocol will be used to retrieve the information or access the object.

The URL describes where something is and how it will be retrieved. The how is specified by the protocol (for example, HTTP). The where is specified by the machine name, the directory name, and the filename. Slashes and other characters are used to separate the parts of the address into machine-readable pieces.

The basic structure of the URL is shown here:

protocol://site address/directory/filename

Every Web document exists on some server computer somewhere on the global Internet or within a private intranet. The first step in finding a document is to identify its server. The most convenient way to do this on a TCP/IP-based network is with a symbolic name, called a domain name

Domain Type	Domain Description	Example
com	Commercial entities and individuals	apple.com
net	Networks and network providers	cerf.net
org	Nonprofit and other organizations	greenpeace.org
edu	Four-year colleges and universities	ucla.edu
gov	United States federal government agencies	whitehouse.gov
mil	United States federal government military entities	nosc.mil
us	Used for a variety of organizations and individuals, including K through 12 education, libraries, and city and county governments	co.san-diego.ca.us

Table 4-1. *Domain Types in the United States*

Web Server:

For International Network communication, we require a web browser and web servers. Web browser and servers play an important role to establish the connection. The client sends requests for web document or service. The message goes from the web browser to the web server is known as an HTTP request. When the web server receives the request, it searches its stores to find the appropriate page. If the web server is able to locate the page, it parcels up to the HTML contained within (using some transport layer protocol), addresses these parcels to the browser (using HTTP), and transmit them back across the network.
If the web server is unable to find the requested page, it sends a page containing an error

message (i.e. Error 404 – page not found) and it parcels up to the dispatches that page to the browser. This message received by the web browser by the server is called the HTTP response.

Functions of web servers:

- Web server is a program or the computer that provides services to other programs called client.
- The Web server accepts, approve and respond to the request made by the web browser for a web document or services.
- The web server is software or a system which maintains the web applications, generate response and accept clients data.
- The web server gets HTTP requests and sends HTTP responses.
- There exist three types of processing models for web server i.e Process-based, Thread based and Hybrid.
- Web servers provide an area to store and organize the pages of the website.
- The web server can be a remote machine placed at the other side of your network or even on the other end of the globe, or it is your very own personal computer at home.

Following are the popular web servers:

- Apache HTTP Server
- Internet Information Services
- Sun Java System Web Server
- Jigsaw Server

WEB Protocols:

HTTP

A minimal HTTP URL simply gives a server name. It provides no directory or file information. A minimal HTTP formula commonly occurs for corporate addresses used in advertising:

■ Formula http://server/

■ Example http://www.bigcompany.com/

A minimal HTTP URL implicitly requests the home directory of a Web site. Even when a trailing slash isn't used, it is assumed and added either by the user agent or the Web server, making an address such as http://www.bigcompany.com become http://www.bigcompany.com/. By default, requesting a directory often results in the server returning a default file from the directory, termed the index file. Usually, index files are named index.htm or default.htm (or index.html and default.html, respectively), depending on the server software being used. This is only a convention; Web administrators are free to name default index files whatever they like. Interestingly, many people put special importance on the minimal HTTP URL form when, like all other file-retrieval URLs, this form simply specifies a particular directory or default index file to return, although this isn't always explicitly written out.

The Hypertext Transfer Protocol (HTTP) is an application-level protocol for distributed, collaborative, hypermedia information systems. This is the foundation for data communication for the World Wide Web (i.e. internet) since 1990. HTTP is a generic and stateless protocol which can be used for other purposes as well using extensions of its request methods, error codes, and headers.

Basically, HTTP is a TCP/IP based communication protocol, that is used to deliver data (HTML files, image files, query results, etc.) on the World Wide Web. The default port is TCP 80, but other ports can be used as well. It provides a standardized way for computers to communicate with each other. HTTP specification specifies how clients' request data will be constructed and sent to the server, and how the servers respond to these requests.

Basic Features

There are three basic features that make HTTP a simple but powerful protocol:

- HTTP is connectionless: The HTTP client, i.e., a browser initiates an HTTP request and after a request is made, the client waits for the response. The server processes the request and sends a response back after which client disconnect the connection. So client and server knows about each other during current request and response only. Further requests are made on new connection like client and server are new to each other.

- HTTP is media independent: It means, any type of data can be sent by HTTP as long as both the client and the server know how to handle the data content. It is required for the client as well as the server to specify the content type using appropriate MIME-type.
- HTTP is stateless: As mentioned above, HTTP is connectionless and it is a direct result of HTTP being a stateless protocol. The server and client are aware of each other only during a current request. Afterwards, both of them forget about each other. Due to this nature of the protocol, neither the client nor the browser can retain information between different requests across the web pages.

FTP

The File Transfer Protocol, which predates the browser-oriented HTTP protocol, transfers files to and from a server. It generally is geared toward transferring files that are to be locally stored rather than immediately viewed. A browser may allow files to be viewed immediately. Today, because of its efficiency, FTP is most commonly used to download large files, such as complete applications. These URLs share with HTTP the formula for indicating a server, port, directory, and file:

■ Formula ftp://server:port/directory/file

■ Example ftp://ftp.bigcompany.com:9978/info/somefile.exe

A minimal FTP URL specifies a server and then lists the following directory: ftp://ftp.bigcompany.com. Generally, however, FTP URLs are used to access by name and directory a particular file in an archive, as shown in this formula:

■ Formula ftp://server/directory path/file

■ Example ftp://ftp.bigcompany.com/info/somefile.exe

FTP is an authenticated protocol, which means that every valid FTP request requires a defined user account on the server downloading the files. In practice, many FTP resources are intended for general access, and defining a unique account for every potential user is impractical.

telnet

The telnet protocol allows a user to open an interactive terminal session on a remote host computer. A minimal telnet URL, shown next, simply gives the remote system's name. After a connection is made, the system prompts for an account name and password.

■ Formula telnet://server

■ Example telnet://host.bigcompany.com

As an authenticated protocol, telnet generally requires a defined user account on the remote system. When this is unspecified, the user agent or helper application handling telnet prompts for such information. Like FTP, a telnet URL can also contain

an account name and password as parameters. But, as with FTP URLs, be careful about including passwords in public access documents, such as HTML files on the Web. Because of the risk of password interception, the password is optional in the formula:

■ Formula telnet://user:password@server

■ Example telnet://jsmith:harmony@host.bigcompany.com

Anchor tag <a>:

A webpage can contain various links that take you directly to other pages and even specific parts of a given page. These links are known as hyperlinks.

Hyperlinks allow visitors to navigate between Web sites by clicking on words, phrases, and images. Thus you can create hyperlinks using text or images available on a webpage.

Linking Documents

A link is specified using HTML tag <a>. This tag is called **anchor tag** and anything between the opening <a> tag and the closing </a> tag becomes part of the link and a user can click that part to reach to the linked document. Following is the simple syntax to use <a> tag.

<a href = "Document URL" ... attributes-list>Link Text</a>

```
<!DOCTYPE html>
```

```
<html>

  <head>
    <title>Hyperlink Example</title>
  </head>

  <body>
    <p>Click following link</p>
    <a href = "https://www.tutorialspoint.com" target = "_self">Tutorials Point</a>
  </body>

</html>
```

This will produce the following result, where you can click on the link generated to reach to the home page of Tutorials Point (in this example).

Click following link

Tutorials Point

The target Attribute

We have used **target** attribute in our previous example. This attribute is used to specify the location where linked document is opened. Following are the possible options −

Sr.No	Option & Description
1	**_blank** Opens the linked document in a new window or tab.
2	**_self** Opens the linked document in the same frame.
3	**_parent** Opens the linked document in the parent frame.

4	**_top** Opens the linked document in the full body of the window.
5	**Targetframe** Opens the linked document in a named *targetframe*.

HTML Email Tag (mailto:)

HTML **<a>** tag provides you option to specify an email address to send an email. While using <a> tag as an email tag, you will use **mailto: email address** along with *href*attribute. Following is the syntax of using **mailto** instead of using http.

<a href = "mailto: abc@example.com">Send Email</a>

This code will generate the following link which you can use to send email.

Send Email

Now, if a user clicks this link, it launches one Email Client (like Lotus Notes, Outlook Express etc.) installed on your user's computer. There is another risk to use this option to send email because if user do not have email client installed on their computer then it would not be possible to send email.

The Role of Images on the Web

When Mosaic first came out, the ability to view an image within the browser window was a huge improvement. At first, images often were used to show the logo of the company or present a graph. Today, however, some sites seem to be more about the images than about textual information. Web images can be used to illustrate an idea, show strictly visual information, provide navigation, and serve as decoration. The saying **"a picture is worth a thousand words"** is very true when illustrating an idea. Images can be used to show procedures, product applications, design styles, and a variety of other concepts.

Last but not least, images can provide decoration for a site. Pictures make things more interesting, even if they just break up the monotony of page after page of text. A splash of color

and a few images can be pleasing. Too much decoration, however, can turn a page into the online equivalent of an over decorated Christmas tree completes with pink snow flocking.

A good-looking, logical user interface improves a site, while a poorly designed user interface negatively affects the best content. Navigation is important and should look graphically appropriate. Images are most important when they enhance the content of the Web site. An image that provides more information than would be conveyed with plain text is truly valuable. Even if the image significantly increases the download time of the page, the supplementary information provided may be well worth it.

<IMG> Tag

The HTML <img> tag is used to put an image in an HTML document.

```
<!DOCTYPE html>
<html>

  <head>
    <title>HTML  Tag</title>
  </head>

  <body>
    <img src = "https://www.tutorialspoint.com/images/html.gif"
      alt = "HTML Tutorial" height = "150" width = "140" />
  </body>
</html>
```

This will produce the following result –

Images are very important to beautify as well as to depict many complex concepts in simple way on your web page. This tutorial will take you through simple steps to use images in your web pages.

Insert Image

You can insert any image in your web page by using **<img>** tag. Following is the simple syntax to use this tag.

<img src = "Image URL" ... attributes-list/>

The <img> tag is an empty tag, which means that, it can contain only list of attributes and it has no closing tag.

Example

To try following example, let's keep our HTML file test.htm and image file test.png in the same directory −

```
<!DOCTYPE html>
<html>

  <head>
    <title>Using Image in Webpage</title>
  </head>

  <body>
    <p>Simple Image Insert</p>
    <img src = "/html/images/test.png" alt = "Test Image" />
  </body>

</html>
```

This will produce the following result –

You can use PNG, JPEG or GIF image file based on your comfort but make sure you specify correct image file name in **src** attribute. Image name is always case sensitive.

The **alt** attribute is a mandatory attribute which specifies an alternate text for an image, if the image cannot be displayed.

Set Image Location

Usually we keep all the images in a separate directory. So let's keep HTML file test.htm in our home directory and create a subdirectory **images** inside the home directory where we will keep our image test.png.

Example

Assuming our image location is "image/test.png", try the following example −

```
<!DOCTYPE html>
<html>

  <head>
    <title>Using Image in Webpage</title>
  </head>

  <body>
    <p>Simple Image Insert</p>
    <img src = "/html/images/test.png" alt = "Test Image" />
  </body>

</html>
```

This will produce the following result –

Set Image Width/Height

You can set image width and height based on your requirement using **width** and **height** attributes. You can specify width and height of the image in terms of either pixels or percentage of its actual size.

```
<!DOCTYPE html>
<html>

  <head>
    <title>Set Image Width and Height</title>
  </head>

  <body>
    <p>Setting image width and height</p>
    <img src = "/html/images/test.png" alt = "Test Image" width = "150" height = "100"/>
  </body>

</html>
```

This will produce the following result –

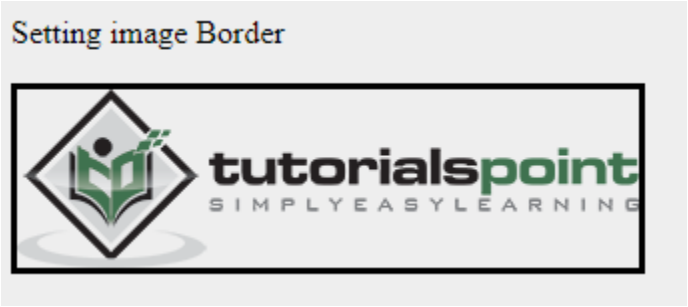
Set Image Border

By default, image will have a border around it, you can specify border thickness in terms of pixels using border attribute. A thickness of 0 means, no border around the picture.

```
<!DOCTYPE html>
<html>

  <head>
    <title>Set Image Border</title>
  </head>
```

```
  <body>
    <p>Setting image Border</p>
    <img src = "/html/images/test.png" alt = "Test Image" border = "3"/>
  </body>

</html>
```

This will produce the following result –



Set Image Alignment

By default, image will align at the left side of the page, but you can use **align** attribute to set it in the center or right.

```
<!DOCTYPE html>
<html>

  <head>
    <title>Set Image Alignment</title>
  </head>

  <body>
    <p>Setting image Alignment</p>
    <img src = "/html/images/test.png" alt = "Test Image" border = "3" align = "right"/>
  </body>

</html>
```

This will produce the following result –

Introduction to Tables

A table represents information in a tabular way, like a spreadsheet: distributed across a grid of rows and columns. Web-based tables have also become a common way to create document layout and design. But unlike printed tables, HTML tables can contain information that is dynamic, or interactive, such as the results of a database query. A table is dynamically generated using the template and the results of a particular database query. Taken together, these capabilities make tables one of HTML's most useful and sophisticated resources.

HTML represents a basic table using four elements.

In HTML, a table, represented by <TABLE>…..</TABLE> tag

In HTML, a heading, represented by <TH>…….</TH> tag

In HTML, a row, represented by <TR>…..</TR> tag

In HTML, a data, represented by <TD>…..</TD> tag

Syntax:

```
<!DOCTYPE html>
<html>

  <head>
    <title>HTML Table Header</title>
  </head>

  <body>
    <table border = "1">
      <tr>
        <th>Name</th>
        <th>Salary</th>
      </tr>
      <tr>
        <td>Ramesh Raman</td>
        <td>5000</td>
      </tr>
```

```
      <tr>
        <td>Shabbir Hussein</td>
        <td>7000</td>
      </tr>
    </table>
  </body>

</html>
```

This will produce the following result −

Name	Salary
Ramesh Raman	5000
Shabbir Hussein	7000

Cellpadding and Cellspacing Attributes

There are two attributes called *cellpadding* and *cellspacing* which you will use to adjust the white space in your table cells. The cellspacing attribute defines space between table cells, while cellpadding represents the distance between cell borders and the content within a cell.

```
<!DOCTYPE html>
<html>

  <head>
    <title>HTML Table Cellpadding</title>
  </head>

  <body>
    <table border = "1" cellpadding = "5" cellspacing = "5">
      <tr>
        <th>Name</th>
        <th>Salary</th>
      </tr>
      <tr>
        <td>Ramesh Raman</td>
        <td>5000</td>
      </tr>
      <tr>
```

```
        <td>Shabbir Hussein</td>
        <td>7000</td>
      </tr>
    </table>
  </body>

</html>
```

This will produce the following result –

Name	Salary
Ramesh Raman	5000
Shabbir Hussein	7000

Colspan and Rowspan Attributes

You will use **colspan** attribute if you want to merge two or more columns into a single column.

Similar way you will use **rowspan** if you want to merge two or more rows.

```
<!DOCTYPE html>
<html>

  <head>
    <title>HTML Table Colspan/Rowspan</title>
  </head>

  <body>
    <table border = "1">
      <tr>
        <th>Column 1</th>
        <th>Column 2</th>
        <th>Column 3</th>
      </tr>
      <tr>
        <td rowspan = "2">Row 1 Cell 1</td>
        <td>Row 1 Cell 2</td>
        <td>Row 1 Cell 3</td>
      </tr>
      <tr>
```

```
        <td>Row 2 Cell 2</td>
        <td>Row 2 Cell 3</td>
      </tr>
      <tr>
        <td colspan = "3">Row 3 Cell 1</td>
      </tr>
    </table>
  </body>

</html>
```

This will produce the following result –

Column 1	Column 2	Column 3
Row 1 Cell 1	Row 1 Cell 2	Row 1 Cell 3
	Row 2 Cell 2	Row 2 Cell 3
Row 3 Cell 1		

Table Height and Width

You can set a table width and height using **width** and **height** attributes. You can specify table width or height in terms of pixels or in terms of percentage of available screen area.

```
<!DOCTYPE html>
<html>

  <head>
    <title>HTML Table Width/Height</title>
  </head>

  <body>
    <table border = "1" width = "400" height = "150">
      <tr>
        <td>Row 1, Column 1</td>
        <td>Row 1, Column 2</td>
      </tr>

      <tr>
        <td>Row 2, Column 1</td>
        <td>Row 2, Column 2</td>
      </tr>
    </table>
  </body>
```

```
</html>
```

This will produce the following result −

Row 1, Column 1	Row 1, Column 2
Row 2, Column 1	Row 2, Column 2

Tables Backgrounds

You can set table background using one of the following two ways −

- **bgcolor** attribute − You can set background color for whole table or just for one cell.
- **background** attribute − You can set background image for whole table or just for one cell.

You can also set border color also using **bordercolor** attribute.

```
<!DOCTYPE html>
<html>

  <head>
    <title>HTML Table Background</title>
  </head>

  <body>
    <table border = "1" bordercolor = "green" bgcolor = "yellow">
      <tr>
        <th>Column 1</th>
        <th>Column 2</th>
        <th>Column 3</th>
      </tr>
      <tr>
        <td rowspan = "2">Row 1 Cell 1</td>
        <td>Row 1 Cell 2</td>
        <td>Row 1 Cell 3</td>
      </tr>
      <tr>
        <td>Row 2 Cell 2</td>
```

```
        <td>Row 2 Cell 3</td>
      </tr>
      <tr>
        <td colspan = "3">Row 3 Cell 1</td>
      </tr>
    </table>
  </body>

</html>
```

This will produce the following result –

Column 1	Column 2	Column 3
Row 1 Cell 1	Row 1 Cell 2	Row 1 Cell 3
	Row 2 Cell 2	Row 2 Cell 3
Row 3 Cell 1		

Here is an example of using **background** attribute. Here we will use an image available in /images directory.

```
<!DOCTYPE html>
<html>

  <head>
    <title>HTML Table Background</title>
  </head>

  <body>
    <table border = "1" bordercolor = "green" background = "/images/test.png">
      <tr>
        <th>Column 1</th>
        <th>Column 2</th>
        <th>Column 3</th>
      </tr>
      <tr>
        <td rowspan = "2">Row 1 Cell 1</td>
        <td>Row 1 Cell 2</td><td>Row 1 Cell 3</td>
      </tr>
      <tr>
        <td>Row 2 Cell 2</td>
        <td>Row 2 Cell 3</td>
      </tr>
      <tr>
```

```
        <td colspan = "3">Row 3 Cell 1</td>
      </tr>
    </table>
  </body>

</html>
```

This will produce the following result. Here background image did not apply to table's header.

Column 1	Column 2	Column 3
Row 1 Cell 1	Row 1 Cell 2	Row 1 Cell 3
	Row 2 Cell 2	Row 2 Cell 3
Row 3 Cell 1		

The Role of Images on the Web

When Mosaic first came out, the ability to view an image within the browser window was a huge improvement. At first, images often were used to show the logo of the company or present a graph. Today, however, some sites seem to be more about the images than about textual information. Web images can be used to illustrate an idea, show strictly visual information, provide navigation, and serve as decoration. The saying **"a picture is worth a thousand words"** is very true when illustrating an idea. Images can be used to show procedures, product applications, design styles, and a variety of other concepts.

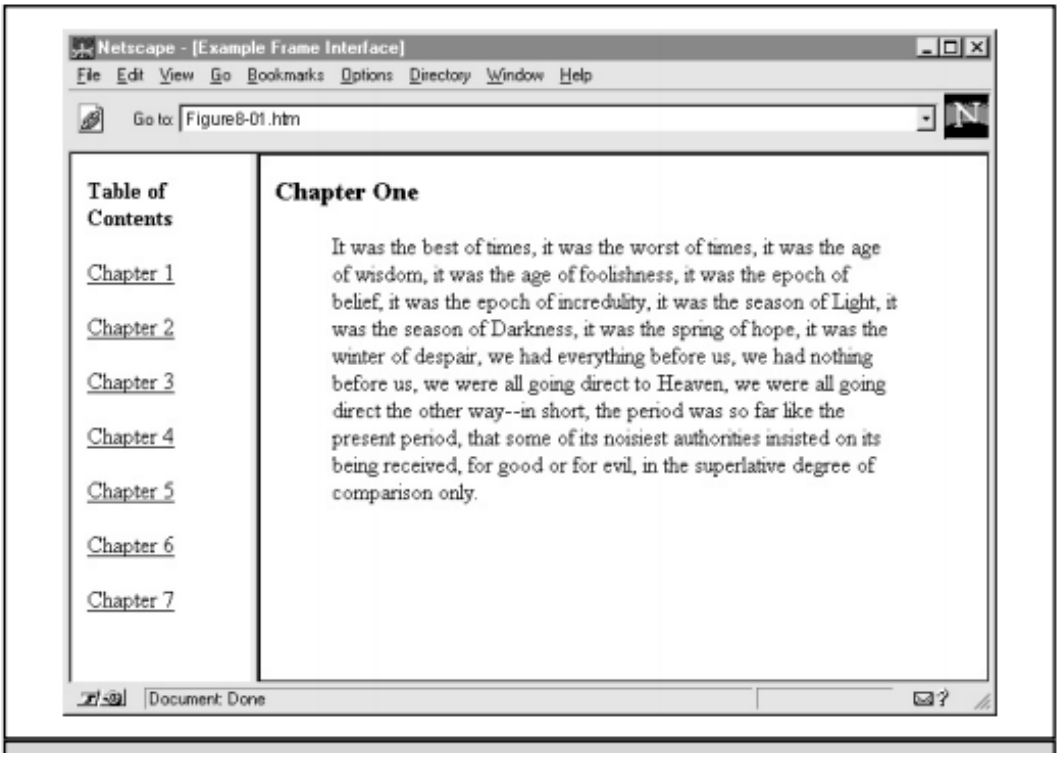
Last but not least, images can provide decoration for a site. Pictures make things more interesting, even if they just break up the monotony of page after page of text. A splash of color and a few images can be pleasing. Too much decoration, however, can turn a page into the online equivalent of an over decorated Christmas tree completes with pink snow flocking.

A good-looking, logical user interface improves a site, while a poorly designed user interface negatively affects the best content. Navigation is important and should look graphically appropriate. Images are most important when they enhance the content of the Web site. An image that provides more information than would be conveyed with plain text is truly valuable. Even if the image significantly increases the download time of the page, the supplementary information provided may be well worth it.

Unit-III Basic Interactivity with DHTML

Frames:

A framed document divides a browser window into multiple panes, or smaller window frames. Each frame may contain a different document. The benefits of this approach are obvious. Users can view information in one frame while keeping another frame open for reference, instead of moving back and forth between pages. The contents of one frame can be manipulated, or linked, to the contents of another. This enables designers to build sophisticated interfaces. For example, one frame can contain links that produce a result in another frame.



Frames offer fixed-screen navigation—whereby certain information is always at the forefront—and other options not found in the single-window approach.

Overview of Frames

HTML frames are used to divide your browser window into multiple sections where each section can load a separate HTML document. A collection of frames in the browser window is known as a frameset. The window is divided into frames in a similar way the tables are organized: into rows and columns.

A frame is an independent scrolling region, or window, of a Web page. Every Web page may be divided into many individual frames, which can even be nested within other frames. Fixed screen sizes limit how many frames can realistically be used simultaneously.

Each frame in a window may be separated from the others with a border; in this way, a framed document may resemble a table. However, frames aren't a fancy form of tables. Each separate frame may contain a different document, indicated by a unique URL.

Disadvantages of Frames

There are few drawbacks with using frames, so it's never recommended to use frames in your web pages −

- Some smaller devices cannot cope with frames often because their screen is not big enough to be divided up.
- Sometimes your page will be displayed differently on different computers due to different screen resolution.
- The browser's *back* button might not work as the user hopes.
- There are still few browsers that do not support frame technology.

Frame Targeting

When using frames, you often may find that making the links in one frame target another frame is beneficial. This way, when a user clicks a button or activates a link in one framed document, the requested page loads in another frame. In the simple frame example in the preceding section, you might want to have the frame named Controls target the frame named Display. Link targeting has two steps:

1. Ensure frame naming by setting the NAME attribute in the element to a unique name.

2. Use the TARGET attribute in the element to set the target for the anchor. For example, a link such as

<A HREF =http://www.google.com Target="Display"> loads the site specified by the HREF into the window called Display, if such a frame exists. If the target specified by the name doesn't exist, the link loads over the window it is in. Some particular values for the TARGET attribute have special meaning, which are summarized in Table 8-1.

Value	Meaning
_blank	Load the page into a new, generally unnamed, window.
_self	Load the page over the current frame.
_parent	Load the link over the parent frame.
_top	Load the link over all the frames in the window.

Table 8-1. *Reserved **TARGET** Values*

Creating Frames

To use frames on a page we use <frameset> tag instead of <body> tag. The <frameset> tag defines, how to divide the window into frames. The **rows** attribute of <frameset> tag defines horizontal frames and **cols** attribute defines vertical frames. Each frame is indicated by <frame> tag and it defines which HTML document shall open into the frame.

```
<!DOCTYPE html>
<html>

   <head>
      <title>HTML Frames</title>
   </head>
```

```
   <frameset rows = "10%,80%,10%">
      <frame name = "top" src = "/html/top_frame.htm" />
      <frame name = "main" src = "/html/main_frame.htm" />
      <frame name = "bottom" src = "/html/bottom_frame.htm" />


   </frameset>

</html>
```

Sr.No	Attribute & Description
1	**Cols** Specifies how many columns are contained in the frameset and the size of each column. You can specify the width of each column in one of the four ways − Absolute values in pixels. For example, to create three vertical frames, use *cols = "100, 500, 100"*. A percentage of the browser window. For example, to create three vertical frames, use *cols = "10%, 80%, 10%"*.
2	**Rows** This attribute works just like the cols attribute and takes the same values, but it is used to specify the rows in the frameset. For example, to create two horizontal frames, use *rows = "10%, 90%"*. You can specify the height of each row in the same way as explained above for columns.
3	**Border** This attribute specifies the width of the border of each frame in pixels. For example, border = "5". A value of zero means no border.

4	**Frameborder** This attribute specifies whether a three-dimensional border should be displayed between frames. This attribute takes value either 1 (yes) or 0 (no). For example frameborder = "0" specifies no border.
5	**Framespacing** This attribute specifies the amount of space between frames in a frameset. This can take any integer value. example framespacing = "10" means there should be 10 pixels spacing between each frames.

```
   <head>
      <title>HTML Frames</title>
   </head>

   <frameset rows = "10%,80%,10%">
      <frame name = "top" src = "/html/top_frame.htm" />
      <frame name = "main" src = "/html/main_frame.htm" />
      <frame name = "bottom" src = "/html/bottom_frame.htm" />


   </frameset>

</html>
```

Sr.No	Attribute & Description
1	**Src** This attribute is used to give the file name that should be loaded in the frame. Its value can be any URL. For example, src = "/html/top_frame.htm" will load an HTML file available in html directory.
2	**Name** This attribute allows you to give a name to a frame. It is used to indicate which frame a document should be loaded into. This is especially important when you want to create links in one frame that load pages into an another frame, in which case the second frame needs a name to identify itself as the target of the link.
3	**Frameborder** This attribute specifies whether or not the borders of that frame are shown; it overrides the value given in the frameborder attribute on the <frameset> tag if one is given, and this can take values either 1 (yes) or 0 (no).
4	**Marginwidth** This attribute allows you to specify the width of the space between the left and right of the frame's borders and the frame's content. The value is given in pixels. For example marginwidth = "10".
5	**Marginheight** This attribute allows you to specify the height of the space between the top and bottom of the frame's borders and its contents. The value is given in pixels. For example marginheight = "10".
6	**Noresize** By default, you can resize any frame by clicking and dragging on the borders of a frame. The noresize attribute prevents a user from being able to resize the frame. For example noresize = "noresize".
7	**Scrolling** This attribute controls the appearance of the scrollbars that appear on the frame. This takes values either "yes", "no" or "auto". For example scrolling = "no" means it should not have scroll bars.

8	**Longdesc** This attribute allows you to provide a link to another page containing a long description of the contents of the frame. For example longdesc = "framedescription.htm"

Form Controls

A form is made up of fields or controls, as well as the markup necessary to structure the form and control its presentation. The controls are the items filled in or manipulated by the user to indicate the state of the form. Form controls include text fields, password fields, multiple-line text fields, pop-up menus, scrolled lists, radio buttons, check boxes, and buttons. The most common element used to specify a form control is the <input> element.

Text Controls

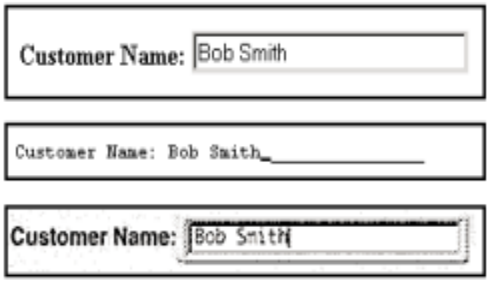
Text controls are form fields, generally one line long, that take text input like a person's name and address, and other information. These fields are specified with the <input> element, but it is possible to specify a multiple-line text field using the <text area> element.

Simple text entry

The simplest type of form control is the text entry type. To set a text entry control, use the <input> element and set the TYPE attribute equal to TEXT:

<Input Type ="text" Name="Customer Name" SIZE="30" MAXLENGTH="60">

All form elements should be named. NAME="Customer Name" is used to create a text field to collect a customer's name on an order form.




```
<!DOCTYPE html>
<html>
<body>
<h2>Text Input</h2>
<form>
  First name:<br>
  <input type="text" name="firstname">
  <br>
  Last name:<br>
  <input type="text" name="lastname">
</form>
<p>Note that the form itself is not visible.</p>
<p>Also note that the default width of a text input field is 20 characters.</p>
</body>
</html>
```

Password Fields

<input> elements of type **password** provide a way for the user to securely enter a password.The element is presented as a one-line plain text editor control in which the text is obscured so that it cannot be read, usually by replacing each character with a symbol such as the asterisk ("*") or a dot ("•").

```
<input type="password" name="pwd">
```

Ex

```
<div>
   <label for="username">Username:</label>
```

```
   <input type="text" id="username" name="username">
</div>

<div>
   <label for="pass">Password (8 characters minimum):</label>
   <input type="password" id="pass" name="password" SIZE="10" MAXLENGTH="10"
         minlength="8" required>
</div>
```

Output

Username:

Password (8 characters minimum):

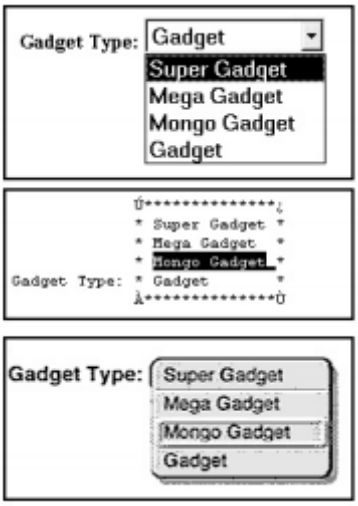
Sign in

Pull-Down Menus

HTML form controls include pull-down menus. A pull-down menu lets the user select one choice out of many possible choices. One nice aspect of pull-down menus is that all choices do not have to be seen on the screen and are normally hidden.

To create a pull-down menu, use the <select> element. This element must include both a start and an end tag. The elements specify the actual choices on the menu, and generally do not use a close tag (similar to how the <Li> element is used). In many ways, the structure of a pull-down menu looks similar to a list structure, as shown in the following code fragment

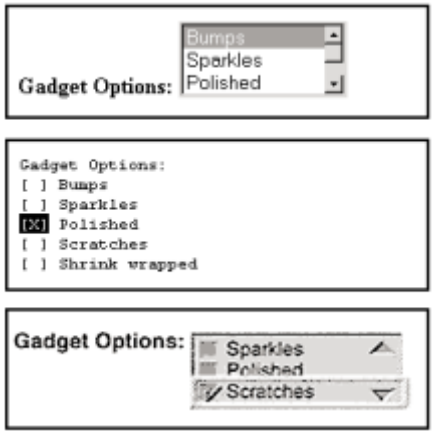
```
<BODY>
<H1 ALIGN="center">Gadget Order Form</H1>

<FORM ACTION="mailto:test@bigcompany.com" METHOD="POST">

<B>Gadget Type:</B>
<SELECT NAME="GadgetType">
    <OPTION VALUE="SG-01">Super Gadget
    <OPTION VALUE="MEG-G5">Mega Gadget
    <OPTION VALUE="MO-45">Mongo Gadget
    <OPTION SELECTED>Gadget
</SELECT>
</FORM>
```



Scrolled Lists

The <select> element also may contain the SIZE attribute, which is used to specify the number of items showing on the screen at once. The default value for this attribute is 1, which specifies a normal pull-down menu. Setting a positive number creates a list in a window of the specified number of rows. In many cases, scrolled lists act just like pull-down menus.

```
<B>Gadget Options:</B>
<SELECT NAME="GadgetOptions" MULTIPLE SIZE="3">
    <OPTION VALUE="Hit with hammer" SELECTED>Bumps
    <OPTION VALUE="Add glitter">Sparkles
    <OPTION VALUE="Buff it">Polished
    <OPTION SELECTED>Scratches
    <OPTION>Shrink wrapped
</SELECT>
</FORM>
<HR>
```



Check Boxes

With the scrolled list, it is possible to select many items out of a large group of items. Unfortunately, not all the items are presented at once for the user to choose. If there are a few options to select from that are not mutually exclusive, it is probably better to use a group of check boxes that the user can check off. Check boxes are best used to toggle choices on and off. While it is possible to have multiple numbers of check boxes and let the user select as many as he or she wants.

```
<!DOCTYPE html>
<html>
<body>
<h1>Show checkboxes:</h1>
<form action="/action_page.php">
```

```
  <input type="checkbox" name="vehicle1" value="Bike"> I have a bike<br>
  <input type="checkbox" name="vehicle2" value="Car"> I have a car<br>
  <input type="checkbox" name="vehicle3" value="Boat" checked> I have a boat<br><br>
  <input type="submit" value="Submit">
</form>
</body>
</html>
```

Radio Buttons

Radio buttons use a similar notation to check boxes, but only one option may be chosen among many. This is an especially good option for choices that don't make sense when selected together. In this sense, radio buttons are like pull-down menus that allow only one choice. The main difference is that all options are shown at once with radio buttons

```
<!DOCTYPE html>
<html>
<body>
<h2>Radio Buttons</h2>
<form>
  <input type="radio" name="gender" value="male" checked> Male<br>
  <input type="radio" name="gender" value="female"> Female<br>
  <input type="radio" name="gender" value="other"> Other
</form>
</body>
</html>
```

Reset and Submit Buttons

Once a form has been filled in, there must be a way to send it on its way, whether it is submitted to a program for processing or simply mailed to an e-mail address. The <input> element has two values, RESET and SUBMIT, for the TYPE attribute; these can create common buttons that are useful for just about any form. Setting the TYPE attribute for the <input> element to RESET creates a button that allows the user to clear or set to default all the form controls at once. Setting the TYPE attribute for <input> to SUBMIT creates a button that triggers the browser to send the contents of the form to the address specified in the ACTION attribute of the
element. Common renderings of the SUBMIT and RESET form controls are shown here:

```
<!DOCTYPE html>
<html>
<body>
<form action="/action_page.php">
  Email: <input type="text" name="email"><br>
  Pin: <input type="text" name="pin" maxlength="4"><br>
  <input type="reset" value="Reset">
  <input type="submit" value="Submit">
</form>
<p>Click on the reset button to reset the form.</p>
</body>
</html>
```

Introduction to DHTML:

- DHTML allows scripting languages to change variables in a web page's definition language, which in turn affects the look and function of otherwise "static" HTML page content, after the page has been fully loaded and during the viewing process. Thus the dynamic characteristic of DHTML is the way it functions while a page is viewed, not in its ability to generate a unique page with each page load.
- Dynamic HTML or DHTML, is a collection of technologies used together to create interactive and animated web sites by using a combination of a HTML and CSS or JavaScript.
- DHTML allows authors to add effects to their pages that are otherwise difficult to achieve. For example, DHTML allows the page author to:
- Animate text and images in their document, independently moving each element from any starting point to any ending point, following a predetermined path or one chosen by the user.
- Embed a ticker that automatically refreshes its content with the latest news, stock quotes, or other data.
- Use a form to capture user input, and then process and respond to that data without having to send data back to the server.
- Include rollover buttons or drop-down menus.

Ramifications of DHTML:

DHTML raises the requirements for Web developers significantly. First, to use DHTML, you need to make sure your pages are coded properly in HTML. Since you have been reading this book, you have a huge head start, because you have been quoting attributes, naming elements, and generating your HTML in a structured fashion. Second, you need to understand style sheets.

This isn't terribly difficult either. Third, you need to understand JavaScript very well. JavaScript isn't a difficult language; but when all is said and done, those who know fundamental programming produce much better code than those who learn programming for the first time with

Java Script. It is sad to see just how many Web scripts don't exhibit fundamental coding techniques, such as variable hiding, modularity, and even simple commenting and logical variable naming. It's no wonder script code breaks all the time. Finally, you need to understand the Document Object Model. This isn't difficult either, particularly if you think about HTML in a structured manner. If you consider all of the things you need to know to create a DHTML page, you realize that creating a DHTML page is much more complicated than creating a static HTML page. With DHTML, we have definitely moved from a page paradigm to a program paradigm, and with that comes new complications.

Rollover Buttons

The preceding examples demonstrate how scripting can be used to manipulate the look of a link as a user rolls over it. A feature available since Netscape called dynamic buttons, commonly called rollover buttons, is one of the first common examples of dynamic page manipulation using JavaScript, although it predates DHTML. We will first explore traditional JavaScript rollovers and then take a look at their more advanced DHTML cousins. A rollover button is a button that becomes active when the user positions the mouse over it. The button may also have a special activation state when it is pressed. To create a rollover button, you will first need at least two, perhaps even three images, to represent each of the button's states—inactive, active, and unavailable. A sample set of rollover images is shown here:

In order to add this rollover image to the page, simply use the <IMG> tag like another image. The idea is to swap the image out when the mouse passes over the image and switch back to the original image when the mouse leaves the image. By literally swapping the value of the SRC attribute, you can achieve the rollover effect. This is in essence what the following script, which should work in Netscape 3 and Internet Explorer 4, would do:

Unit-IV CSS and Java Script

Introduction to CSS

Cascading **S**tyle **S**heets, fondly referred to as **CSS**, is a simply designed language intended to simplify the process of making web pages presentable. CSS allows you to apply styles to web pages. More importantly, CSS enables you to do this independent of the HTML that makes up each web page.

CSS is easy to learn and understood but it provides powerful control over the presentation of an HTML document.

WHY CSS?

- **CSS saves time :** You can write CSS once and reuse same sheet in multiple HTML pages.
- **Easy Maintainence :** To make a global change simply change the style, and all elements in all the webpages will be updated automatically.
- **Search Engines :** CSS is considered as clean coding technique, which means search engines won't have to struggle to "read" its content.
- **Superior styles to HTML :** CSS has a much wider array of attributes than HTML, so you can give a far better look to your HTML page in comparison to HTML attributes.
- **Offline Browsing :** CSS can store web applications locally with the help of offline catche.Using of this we can view offline websites.

Types of Style Sheets

There are 3 types of style sheets. The difference is where the style is defined and the area where that style is applied.

1. 1. **Inline** style sheet within a tag. Applies only to that particular occurrence of that tag.
2. 2. **Internal** (also called **Embedded**) style sheet is defined within the **head section** of a page. Applies to that page only

3. 3. **External** style sheet defined in a separate, hence external, file. Applies to all pages that link to the external style sheet.

Note that more than one of these 3 types can be used on a page, and you can link more than one External style sheet to a page.

1. **Inline style sheet**

Inline style sheet are should only be used **where a particular style is not going to be repeated elsewhere on the page/site**.

The definition is defined within the **HTML tag** in the body section of the HTML code. It must be redefined every time it is required

Syntax

The syntax for inline styles is slightly simpler than that of Internal and External styles in that there is no selector and no curly brackets.

```
<element STYLE="property:value">
```

2. Internal style sheet

The definition is **written once in In the head part of a page**. It must be written on every page that requires that style.Because of the above it is ideal if only **1 page is going to be used for this style**.The styles can then be used more than once throughout the page.

The **Internal** style sheet is defined within the head section.

```
<head>
< style type="text/css">
Your Style definitions go here
< /style>
< /head>
```

Example:

```
<head>
< style type="text/css">
Body {background-color: #3333FF; color: #000033;}
p {margin-left: 6px}
< /style>
< /head>
```

External CSS:

External CSS contains separate CSS file which contains only style property with the help of tag attributes (For example class, id, heading, … etc). CSS property written in a separate file with .css extension and should be linked to the HTML document using **link** tag. This means that for each element, style can be set only once and that will be applied across web pages.

Introduction to JavaScript

Java script is a dynamic computer programming language. It is lightweight and most commonly used as a part of web pages, whose implementations allow client-side script to interact with the user and make dynamic pages. It is an interpreted programming language with object-oriented capabilities. JavaScript was first known as LiveScript, but Netscape changed its name to JavaScript, possibly because of the excitement being generated by Java. JavaScript made its first appearance in Netscape 2.0 in 1995 with the name LiveScript. The general-purpose core of the language has been embedded in Netscape, Internet Explorer, and other web browsers.

The ECMA-262 Specification defined a standard version of the core JavaScript language.

JavaScript is a lightweight, interpreted programming language

.• Designed for creating network-centric applications

.• Complementary to and integrated with Java

.• Complementary to and integrated with HTML

.• Open and cross-platform.

Client-Side JavaScript

Client-side JavaScript is the most common form of the language. The script should be included in or referenced by an HTML document for the code to be interpreted by the browser. It means that a web page need not be a static HTML, but can include programs that interact with the user, control the browser, and dynamically create HTML content. The JavaScript client-side mechanism provides many advantages over traditional CGI server-side scripts. For example, you might use JavaScript to check if the user has entered a valid e-mail address in a form field.

The JavaScript code is executed when the user submits the form, and only if all the entries are valid, they would be submitted to the Web Server. JavaScript can be used to trap user-initiated events such as button clicks, link navigation, and other actions that the user initiates explicitly or implicitly.

Advantages of JavaScript

The merits of using JavaScript are:

- Less server interaction: You can validate user input before sending the page off to the server. This saves server traffic, which means less load on your server.
- Immediate feedback to the visitors: They don't have to wait for a page reload to see if they have forgotten to enter something.
- Increased interactivity: You can create interfaces that react when the user hovers over them with a mouse or activates them via the keyboard.
- Richer interfaces: You can use JavaScript to include such items as drag-anddrop• components and sliders to give a Rich Interface to your site visitors.

Limitations of JavaScript

We cannot treat JavaScript as a full-fledged programming language. It lacks the following important features:

- Client-side JavaScript does not allow the reading or writing of files. This has been kept for security reason.
- JavaScript cannot be used for networking applications because there is no such support available. JavaScript doesn't have any multithreading or multiprocessor capabilities. Once again, JavaScript is a lightweight, interpreted programming language that allows you to build interactivity into otherwise static HTML pages.

JavaScript Data types:

One of the most fundamental characteristics of a programming language is the set of data types it supports. These are the type of values that can be represented and manipulated in a programming language. JavaScript allows you to work with three primitive data types:

- Numbers, e.g., 123, 120.50 etc.
- Strings of text, e.g. "This text string" etc.
- Boolean, e.g. true or false.

JavaScript also defines two trivial data types, null and undefined, each of which defines only a single value. In addition to these primitive data types, JavaScript supports a composite data type known as object. We will cover objects in detail in a separate chapter.

Note: Java does not make a distinction between integer values and floating-point values. All numbers in JavaScript are represented as floating-point values. JavaScript represents numbers using the 64-bit floating-point format defined by the IEEE 754 standard.

JavaScript Variables

Like many other programming languages, JavaScript has variables. Variables can be thought of as named containers. You can place data into these containers and then refer to the data simply by naming the container. Before you use a variable in a JavaScript program, you must declare it. Variables are declared with the **var** keyword as follows.

<Script type =”text/javascript”>
<! –

```
Var money;
Var name;
</script>
```

You can also declare multiple variables with the same var keyword as follows:

```
<Script type =”text/javascript”>
<! –
Var money, name;
</script>
```

Storing a value in a variable is called variable initialization. You can do variable initialization at the time of variable creation or at a later point in time when you need that variable. For instance, you might create a variable named money and assign the value 2000.50 to it later. For another variable, you can assign a value at the time of initialization as follows.

```
<Script type =”text/javascript”>
<! –
Var name =”FYBSC”;
Var money;
Money=150.25;
</script>
```

Note: Use the var keyword only for declaration or initialization, once for the life of any variable name in a document. You should not re-declare same variable twice.

JavaScript is untyped language. This means that a JavaScript variable can hold a value of any data type. Unlike many other languages, you don't have to tell JavaScript during variable declaration what type of value the variable will hold. The value type of a variable can change during the execution of a program and JavaScript takes care of it automatically.

JavaScript Variable Scope

The scope of a variable is the region of your program in which it is defined. JavaScript variables have only two scopes.

- **Global Variables:** A global variable has global scope which means it can be defined anywhere in your JavaScript code.
- **Local Variables:** A local variable will be visible only within a function where it is defined. Function parameters are always local to that function.

Within the body of a function, a local variable takes precedence over a global variable with the same name. If you declare a local variable or function parameter with the same name as a global variable, you effectively hide the global variable. Take a look into the following example.

```
<script type="text/javascript">
<!--
var myVar = "global"; // Declare a global variable
function checkscope( ) {
   var myVar = "local";  // Declare a local variable
   document.write(myVar);
}
//-->
</script>
```

JavaScript Variable Names

While naming your variables in JavaScript, keep the following rules in mind.

- You should not use any of the JavaScript reserved keywords as a variable name. These keywords are mentioned in the next section. For example, break or Boolean variable names are not valid.
- JavaScript variable names should not start with a numeral (0-9). They must begin with a letter or an underscore character.

  For example, 123test is an invalid variable name but _123test is a valid one.

- JavaScript variable names are case-sensitive. For example, Name and name are two different variables.

Adding script to document

```
<html>
<body>
<pre>
<script type="text/javascript">
document.writeln("Hello World!");
</script>
</pre>
</body>
</html>
```

Program output:

Hello World!

Input and Output Statements

In order for a computer program to perform any useful work, it must be able to communicate with the outside world. The process of communicating with the outside world is known as input/output or I/O. Watson JavaScript includes two output statements for displaying information on the computer's screen, and two input statements – one for reading in text input, the other for reading in numeric input. Most imperative languages include mechanisms for performing other kinds of I/O such as detecting where the mouse is pointing and accessing the contents of a disk drive.

The two output statements included in JavaScript are **document.write** and **document.writeln**. These statements are of the general form:

- **document.write(*expression*);**

- **document.writeln(*expression*);**

where an “expression” can be a constant, a variable, a function call, or a collection of these things connected by appropriate operators. Both the document.write and document.writeln statements display the value of an expression on the computer screen. The only difference between the two is that document.writeln starts a new line after printing the value on the screen. For this reason, document.writeln is pronounced as if it were written “document write line”.

- **Program definition:**

```
<html>
<body>
<pre>
<script type="text/javascript">
    document.writeln("Hello World!");
</script>
</pre>
</body>
</html>
```

- **Program output:**

```
Hello World!
```